1. **An application program in a computer with cache uses 1400 instruction fetch from cache and 100 from main memory. What is hit ratio?**

Hit Ratio = No. of Instruction fetch from cache / Total no. of instructions
= 1400 / 1500 = 93.333%

2. **Consider a two level cache with access time 5 nsec and 80 nsec respectively. If the hit ratio are 95% and 75% respectively in the two caches and main memory access time is 250 nsec. What is the effective access time? (effective access time =H1T1+H2(1-H1)T2+…….**

Access Time = H1T1 + H2 (1 – H1)T2 + (1 – H1)(1 – H2)T3
= .95(5) + (1 - .95).75 X 80 +(1-.95) (1-.75)250
= 4.75 + 3.00 + 3.125 = 10.875 nsec

3. **Let's say that we have two levels of cache, backed by DRAM:**
- L1 cache costs 1 cycle to access and has miss rate of 10%
- L2 cache costs 10 cycles to access and has miss rate of 2%
- DRAM costs 80 cycles to access (and has miss rate of 0%)
Then what would be average memory access time (AMAT)?

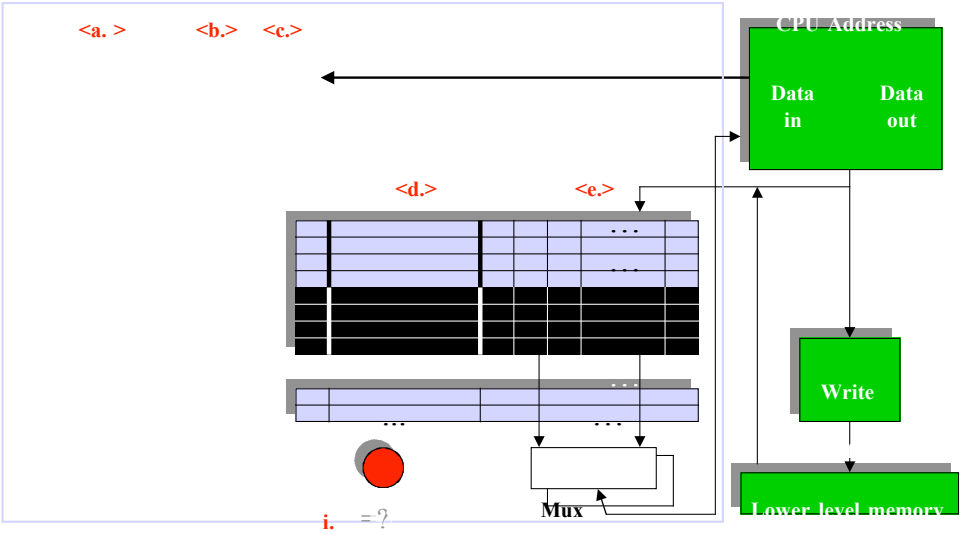
0.90 * 1 +	probability hit in L1 cache * time to access L1
0.10 *	probability miss in L1 cache *
0.98 * (10 + 1)	probability hit in L2 cache * time to access L1 then L2
0.10 *	probability miss in L1 cache *
0.02 *	probability miss in L2 cache *
(1 + 10 + 80)	time to access L1 then L2 then DRAM
= 2.16 cycles	

.

4. You have been asked to design a cache with the following properties:
- Data words are 32 bits each
- A cache block will contain 2048 bits of data
- The cache is direct mapped

- The address supplied from the CPU is 32 bits long
- There are 2048 blocks in the cache
- Addresses are to the word

Pictured below is the general structure of a cache. There are 8 fields (labeled a, b, c, d, e, f, g, and h). In the space below you'll need to indicate the proper name or number of bits for a particular portion of this cache configuration. Whether a name or number should be entered will be specified:



Suggestion – you don't have to fill in the answers "in order". In fact, you need to determine the number of bits associated with the index and offset for example before determining the number of bits in the tag. I'll present the answer this way:

f. (name) _____

- You are being asked to show what part of a physical address form the index, offset, and tag. < f > refers to the most significant bits of the address – so this is the tag.

g. (name) _____

- It follows that the next part of the address is the index.

h. (name) _____

- The least significant bits form the offset.

c. (number) _____

- There are 2_{11} bits / block and there are 2_5 bits / word. Thus there are 2_6 words / block so we need 6 bits of offset.

b. (number) _____

- There are 2_{11} blocks and the cache is direct mapped (or "1-way set associative"). Therefore, we need 11 bits of index.

a. (number) _____

- The remaining bits form the tag. Thus, 32 – 6 – 11 → 15 bits of tag.

d. (number) _____

- Field < d > refers to the fact that a tag must be stored in each block. Thus, 15 bits are kept in each block.

e. (number) _____

- Field < e > asks you to specify the total number of bits / block. This is 2048.

i. What 3 things must be compared at ? to determine if the cache entry is useable or not?

- We need to compare the valid bit associated with the block, the tag stored in the block, and the tag associated with the physical address. The tags should be the same and the valid bit should be 1.

j. What is the total size of the cache?

- There are 2048 blocks in the cache and there are 2048 bits / block.
    - There are 8 bits / byte
    - Thus, there are 256 bytes / block
- 2048 blocks x 256 bytes / block → 2_{19} bytes (or 0.5 MB)

5. Now, let's consider what happens if we make our cache 2-way set-associative instead of direct mapped. However, as before, the following still applies:

- Data words are 32 bits each
- A cache block will contain 2048 bits of data
- The address supplied from the CPU is 32 bits long
- There are 2048 blocks in the cache
- Addresses are to the word

Number of bits in offset?

- There are still 6 bits in the offset; data is still word addressed

Number of bits in index?

- We now need one less bit of index because we address to the set

- 2_{11} blocks / 2_1 blocks/set = 2_{10} sets
- (10 bits of index needed)

Number of bits in tag?

- 32 – 6 – 10 = 16 bits.

6. Now, let's consider what happens if we make our cache 4-way set-associative instead of direct mapped. However, as before, the following still applies:

- Data words are 32 bits each
- A cache block will contain 2048 bits of data
- The address supplied from the CPU is 32 bits long
- There are 2048 blocks in the cache
- Addresses are to the word

Number of bits in offset?

- There are still 6 bits in the offset; data is still word addressed

Number of bits in index?

- We now need one less bit of index because we address to the set
  - 2_{11} blocks / 2_2 blocks/set = 2_9 sets
  - (9 bits of index needed)

Number of bits in tag?

- 32 – 6 – 9 = 17 bits.

7. Effective Access Time example: A computer has a single cache (off-chip) with a 2 ns hit time and a 98% hit rate. Main memory has a 40 ns access time. What is the computer's effective access time? If we add an on-chip cache with a .5 ns hit time and a 94% hit rate, what is the computer's effective access time? How much of a speedup does the on-chip cache give the computer?

Answers:
2 ns + .02 * 40 ns = 2.8 ns.

With the on-chip cache, we have .5 ns + .06 * (2 ns + .02 * 40 ns) = .668 ns. The speedup is 2.8 / .668 = 4.2.

8. Virtual memory problem: Assume a computer has on-chip and off-chip caches, main memory and virtual memory. Assume the following hit rates and access times: on-chip cache 95%, 1 ns, off-chip cache 99%, 10 ns, main memory: X%, 50 ns, virtual memory: 100%, 2,500,000 ns. Notice that the on-chip access time is 1 ns. We do now want our effective access time to increase much beyond 1 ns. Assume that an acceptance effective access time is 1.6 ns. What should X be (the percentage of page faults) to ensure that EAT is no worse than 1.6 ns?

Answer: EAT = 1ns + .05 * (10 ns + .01 * (50 ns + (1 – X) * 2,500,000 ns)). Since we want EAT to be no more than 1.6 ns, we solve for X with 1.6 ns = 1ns + .05 * (10 ns + .01 * (50 ns + (1 – X) * 2,500,000 ns)). X = 1 – ((((((1.25 ns – 1 ns) / .05) – 10 ns) / .01) – 50 ns) / 2,500,000). X = 0.99994 = 99.994%. Our miss rate for virtual memory must be no worse than .006%!

9. Cache/Memory Layout: A computer has an 8 GByte memory with 64 bit word sizes. Each block of memory stores 16 words. The computer has a direct-mapped cache of 128 blocks. The computer uses word level addressing. What is the address format? If we change the cache to a 4- way set associative cache, what is the new address format?

Answers:
With 8 GB and a 64 bit word size, there are 8 GB / (8 bytes / word) = 1 GW of memory. This requires 30 bits for an address. Of the 30 bits, we need 4 bits for the word on the line and 7 bits for the block number, leaving 30 – (7 + 4) = 19 bits for the tag. So the address format is 19 – 7 – 4.

If we have a 4-way set associative cache instead, then there will be 4 sets with 128 / 4 = 32 blocks per set. So we would only need 5 bits for the block number, leaving 30 – (5 + 4) = 21 bits for the tag. So the address format is 21 – 5 – 4.

10. Direct Mapping Question: Assume a computer has 32 bit addresses. Each block stores 16 words.
A direct-mapped cache has 256 blocks. In which block (line) of the cache would we look for each of the following addresses? Addresses are given in hexadecimal for convenience.

a. 1A2BC012
b. FFFF00FF
c. 12345678
d. C109D532

Answers: Of the 32 bit address, the last four bits denote the word on the line. Since four bits is used for one hex digit, the last digit of the address is the word on the line. With 256 blocks in the cache, we need 8 bits to denote the block number. This would be the third to last and second to last hex digit.
a. this would be block 01, which is block 1 b.
this would be 0F which is block 15
c. this would be 67 which is block 103 (remember, 67 is a hex value)
d. this would be 53 which is block 83.

11. Assume a program consists of 8 pages and a computer has 16 frames of memory. A page consists of 4096 words and memory is word addressable. Currently, page 0 is in frame 2, page 4 is in frame 15, page 6 is in frame 5 and page 7 is in frame 9. No other pages are in memory. Translate the memory addresses below.
    a. 111000011110000 b.
    000000000000000

Answers: The first three bits denote the page number (8 pages) and we swap it out for the four bit frame number (16 frames).
a. 111 (page 7) becomes 1001 (frame 9) so 111000011110000 becomes 1001000011110000
b. 000 (page 0) becomes 0010 (frame 2) so 000000000000000 becomes 001000000000000

12. A computer system uses 16-bit memory addresses. It has a 2K-byte cache organized in a direct-mapped manner with 64 bytes per cache block. Assume that the size of each memory word is 1 byte.

(a) Calculate the number of bits in each of the Tag, Block, and Word fields of the memory address.
(b) When a program is executed, the processor reads data sequentially from the following word addresses:

128, 144, 2176, 2180, 128, 2176

All the above addresses are shown in decimal values. Assume that the cache is initially empty. For each of the above addresses, indicate whether the cache access will result in a hit or a miss.

Solution

(a) Block size = 64 bytes = 2_6 bytes = 2_6 words (since 1 word = 1 byte)
Therefore, **Number of bits in the *Word* field = 6**
Cache size = 2K-byte = 2_{11} bytes
Number of cache blocks = Cache size / Block size = $2_{11}/2_6 = 2_5$
Therefore, **Number of bits in the *Block* field = 5**
Total number of address bits = 16
Therefore, **Number of bits in the *Tag* field = 16 - 6 - 5 = 5**
For a given 16-bit address, the 5 most significant bits, represent the *Tag*, the next 5 bits represent the *Block*, and the 6 least significant bits represent the *Word*.

(b) The cache is initially empty. Therefore, all the cache blocks are invalid.

Access # 1:

Address = $(128)_{10}$ = $(0000000010000000)_2$
(**Note:** Address is shown as a 16-bit number, because the computer uses 16-bit addresses)
For this address, *Tag* = 00000, *Block* = 00010, *Word* = 000000
Since the cache is empty before this access, this will be a cache **miss**
After this access, **Tag field for cache block 00010 is set to 00000**

Access # 2:
Address = $(144)_{10}$ = $(0000000010010000)_2$
For this address, *Tag* = **00000**, *Block* = **00010**, *Word* = 010000
Since tag field for cache block **00010** is **00000** before this access, this will be a cache **hit** (because address tag = block tag)

Access # 3:
Address = $(2176)_{10}$ = $(0000100010000000)_2$
For this address, *Tag* = **00001**, *Block* = **00010**, *Word* = 000000
Since tag field for cache block **00010** is **00000** before this access, this will be a cache **miss** (address tag ≠ block tag)
After this access, **Tag field for cache block 00010 is set to 00001**

Access # 4:
Address = $(2180)_{10}$ =

For this address, *Tag* = **00001**, *Block* = **00010**, *Word* = 000100
Since tag field for cache block **00010** is **00001** before this access, this will be a cache **hit** (address tag = block tag)

Access # 5:
Address = $(128)_{10}$ = $(0000000010000000)_2$
For this address, *Tag* = **00000**, *Block* = **00010**, *Word* = 000000
Since tag field for cache block **00010** is **00001** before this access, this will be a cache **miss** (address tag ≠ block tag)
After this access, **Tag field for cache block 00010 is set to 00000**

Access # 6:
Address = $(2176)_{10}$ = $(0000100010000000)_2$
For this address, *Tag* = **00001**, *Block* = **00010**, *Word* = 000000
Since tag field for cache block **00010** is **00001** before this access, this will be a cache **miss** (address tag ≠ block tag)
After this access, **Tag field for cache block 00010 is set to 00001**

Cache hit rate = Number of hits / Number of accesses = 2/6 = 0.333

13. Repeat Problem # 1, if the cache is organized as a 2-way set-associative cache that uses the LRU replacement algorithm.

Solution

(a) Block size = 64 bytes = 2_6 bytes = 2_6 words
Therefore, **Number of bits in the *Word* field = 6**
Cache size = 2K-byte = 2_{11} bytes
Number of cache blocks per set = 2
Number of sets = Cache size / (Block size * Number of blocks per set) = $2_{11}/(2_6 * 2) = 2_4$
Therefore, **Number of bits in the *Set* field = 4**
Total number of address bits = 16
Therefore, **Number of bits in the *Tag* field = 16 - 6 - 4 = 6**

(b) The cache is initially empty. Therefore, all the cache blocks are invalid.

Access # 1:
Address = $(128)_{10}$ = $(0000000010000000)_2$
For this address, *Tag* = 000000, *Set* = 0010, *Word* = 000000
Since the cache is empty before this access, this will be a cache **miss**
After this access, **Tag field for the first block in set 0010 is set to 000000**

Access # 4:
Address = $(2112)_{10}$ = $(0000100000000000)_2$
For this address, *Tag* = **000000**, *Set* = **0010**, *Word* = 010000
The tag field for this address matches the tag field for the first block in set 0010. Therefore, this access will be a cache **hit**.

Access # 3:
Address = $(2176)_{10}$ = $(0000100010000000)_2$
For this address, *Tag* = **000010**, *Set*= **0010**, *Word* = 000000
The tag field for this address does not match the tag field for the first block in set 0010. The second block in set 0010 is empty. Therefore, this access will be a cache **miss**.
After this access, **Tag field for the second block in set 0010 is set to 000010**

Access # 4:
Address = $(2180)_{10}$ = $(0000100010000100)_2$
For this address, *Tag* = **000010**, *Set* = **0010**, *Word* = 000100
The tag field for this address matches the tag field for the second block in set 0010. Therefore, this access will be a cache **hit**.

Access # 5:
Address = $(128)_{10}$ = $(0000000010000000)_2$
For this address, *Tag* = **000000**, *Set* = **0010**, *Word* = 000000
The tag field for this address matches the tag field for the first block in set 0010. Therefore, this access will be a cache **hit**.

Access # 6:
Address = $(2176)_{10}$ = $(0000100010000000)_2$
For this address, *Tag* = **000010**, *Set* = **0010**, *Word* = 000000
The tag field for this address matches the tag field for the second block in set 0010. Therefore, this access will be a cache **hit**.

Cache hit rate = Number of hits / Number of accesses = 4/6 = 0.666

<u>Access # 4:</u>

Address = (2180)$_{16}$ =